Practical No. 1

Title: Write a program non-recursive and recursive program to calculate Fibonacci numbers and analyze their time and space complexity.

```
#Non-recursive Fibonacci
nterms = int(input("How many terms? "))
n1, n2 = 0, 1
count = 0
if nterms <= 0:
   print("Please enter a positive integer")
else:
   print("Non-recursive Fibonacci sequence:")
   while count < nterms:
      print(n1, end=" ")
      nth = n1 + n2
      n1 = n2
      n2 = nth
      count += 1
   print()
#Recursive Fibonacci
def fibonacci(n):
   if n <= 1:
      return n
   return fibonacci(n - 1) + fibonacci(n - 2)
if nterms > 0:
   print("Recursive Fibonacci sequence:")
   for i in range(nterms):
      print(fibonacci(i), end=" ")
   print()
print("Non-recursive: Time O(n), Space O(1)")
print("Recursive: Time O(2^n), Space O(n)")
```

Output:

```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/HP/Downloads/1_Fabonacci.py
How many terms? 5
Non-recursive Fibonacci sequence:
0 1 1 2 3
Recursive Fibonacci sequence:
0 1 1 2 3
Non-recursive: Time O(n), Space O(1)
Recursive: Time O(2^n), Space O(n)
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/HP/Downloads/1_Fabonacci.py
How many terms? -4
Please enter a positive integer
Non-recursive: Time O(n), Space O(1)
Recursive: Time O(2^n), Space O(n)
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```